

CoffeeChain — System Architecture

Version: July 2026

Audience: Engineering team, stakeholders, and onboarding

Purpose: End-to-end understanding of what CoffeeChain is, how it is built, and how data flows from supplier dispatch to blockchain-verified farmer receipt.

Table of Contents

1. [Executive Summary](#1-executive-summary)
 2. [High-Level Architecture](#2-high-level-architecture)
 3. [Technology Stack](#3-technology-stack)
 4. [User Roles & Access Control](#4-user-roles--access-control)
 5. [Domain Model](#5-domain-model)
 6. [End-to-End Supply Chain Workflow](#6-end-to-end-supply-chain-workflow)
 7. [Farmer Verification (OTP)](#7-farmer-verification-otp)
 8. [Trust Layer: Receipts, Storage & Blockchain](#8-trust-layer-receipts-storage--blockchain)
 9. [Data Integrity & Tamper Detection](#9-data-integrity--tamper-detection)
 10. [Notifications](#10-notifications)
 11. [Frontend Application](#11-frontend-application)
 12. [Backend API Overview](#12-backend-api-overview)
 13. [Deployment Architecture](#13-deployment-architecture)
 14. [Environment Configuration](#14-environment-configuration)
 15. [Security Summary](#15-security-summary)
 16. [Project Structure](#16-project-structure)
-

1. Executive Summary

CoffeeChain is a **trust and traceability platform** for fertilizer distribution across Tanzania's coffee-producing regions. It connects suppliers, warehouse managers, retailers, cooperatives (AMCOS), regulators, and administrators in a single digital supply chain.

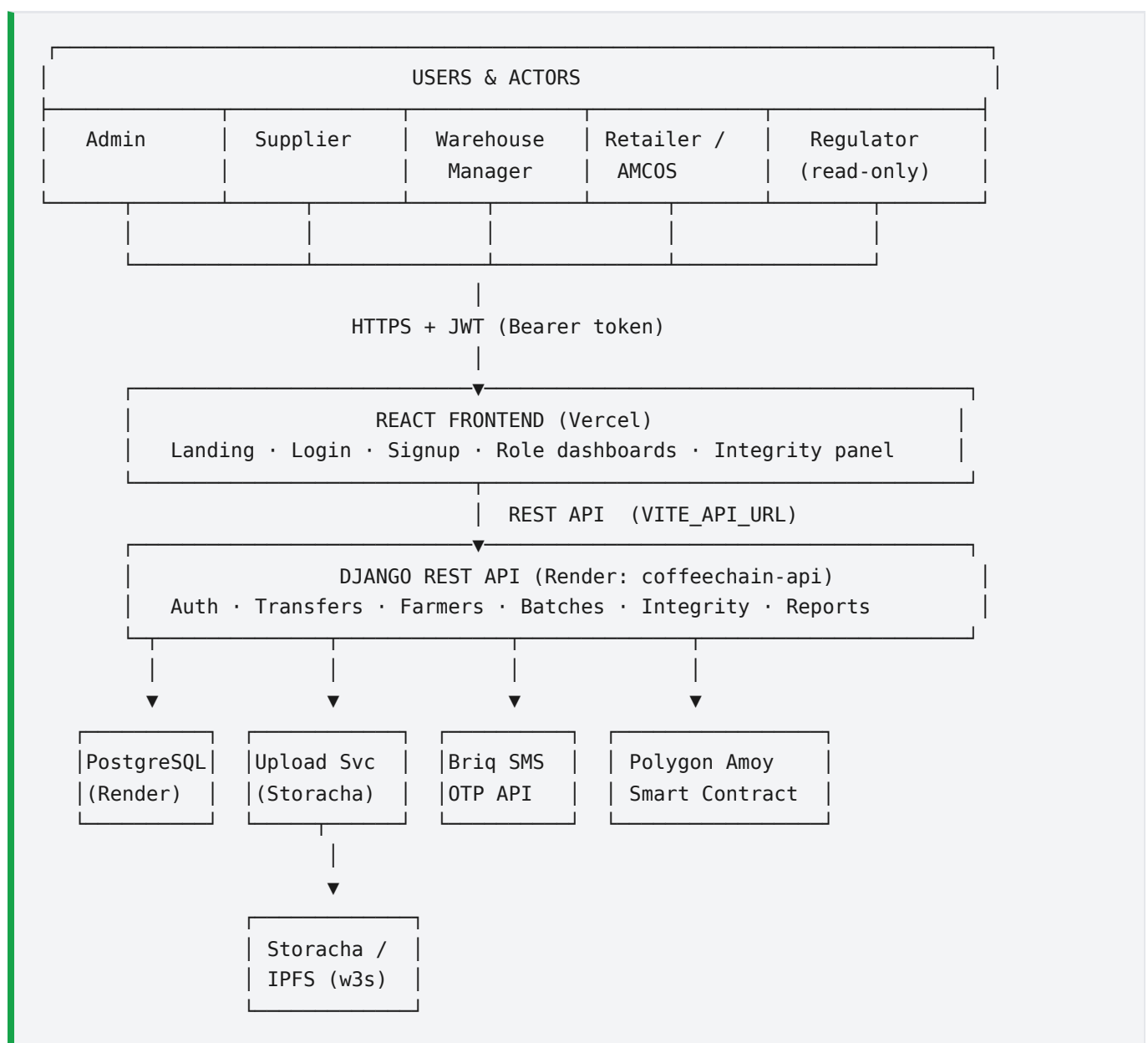
What makes it trustworthy:

- Every farmer delivery can be confirmed with an ****SMS OTP**** (via Briq Karibu).

- A **verification receipt** is stored on **decentralized storage (Storacha/IPFS)** with backend fallback.
- A **cryptographic hash** of the receipt is **anchored on Polygon Amoy** (public blockchain).
- **Integrity monitoring** continuously compares database records against stored receipts and alerts admins on tampering.

Farmers do not log in. They are identified by Ministry of Agriculture IDs and confirm receipt via OTP only.

2. High-Level Architecture



Component Responsibilities

Component	Location	Responsibility
Frontend	`frontend/` → Vercel	User interface, role dashboards, JWT session
Backend API	`backend/` → Render	Business logic, auth, DB, blockchain calls
Upload service	`upload-service/` → Render	Storacha file uploads with fallback
PostgreSQL	Render managed DB	Persistent data, receipts in anchor payload
Smart contract	`smart-contracts/` → Polygon Amoy	Immutable hash anchoring
Briq Karibu	External API	Farmer OTP + admin SMS alerts

3. Technology Stack

Frontend

Technology	Purpose
React 18	UI framework
Vite 6	Build tool
React Router 7	Client-side routing
Tailwind CSS v4	Styling
Radix UI	Accessible components
Recharts	Analytics charts
jsPDF / xlsx	Report export
Sonner	Toast notifications

Backend

Technology	Purpose
Django 5.2	Web framework
Django REST Framework	REST API
SimpleJWT	JWT authentication
PostgreSQL	Production database
Gunicorn	Production WSGI server
web3.py	Polygon blockchain integration
WhiteNoise	Static file serving

Upload Service

Technology	Purpose
Node.js 20	Runtime
Express	HTTP server
@storacha/client	IPFS uploads
Multer	Multipart file handling

Blockchain

Technology	Purpose
Solidity 0.8.20	Smart contract language
Hardhat	Compile & deploy
Polygon Amoy (chainId 80002)	Test network

4. User Roles & Access Control

Authentication Model

CoffeeChain uses **JWT (JSON Web Tokens)** for platform users:

1. User submits username + password → `POST /api/login/`

2. Server returns **access token** (1 hour) + **refresh token** (7 days)
3. Frontend stores tokens in ``localStorage``
4. All API requests include ``Authorization: Bearer <access>``
5. On expiry, frontend silently refreshes; on failure → redirect to login

Public endpoints (no JWT):

- ``POST /api/auth/register/`` — self-service signup (creates pending request, not a live account)
- ``POST /api/internal/receipt-callback/`` — upload-service fallback (shared secret header)

Roles

Role	Django Group	Dashboard	Key Permissions
Admin	<code>`is_staff`</code>	Admin panel	Full CRUD: users, registrations, integrity, reports
Regulator	<code>`Regulator`</code>	Admin panel (read-only)	View users, registrations, integrity — cannot approve or edit
Supplier	<code>`Supplier`</code>	Supplier dashboard	Create batches, dispatch to branches
Warehouse Manager	<code>`WarehouseManager`</code>	Warehouse dashboard	Approve/reject supplier dispatches
Retailer	<code>`Retailer`</code>	Retailer dashboard	Receive stock, sell to farmers, OTP verify
Cooperative (AMCOS)	<code>`Cooperative`</code>	Cooperative dashboard	Register farmers, distribute, OTP verify
Farmer	<code>*(none)*</code>	No login	Identified by Ministry ID; confirms via SMS OTP

Account Creation Paths

Path A – Self registration:

Signup form → PendingRegistration → Admin approves → User + role record created

Path B – Admin creates user:

Admin dashboard → User Management → immediate account

Path C – Demo seed:

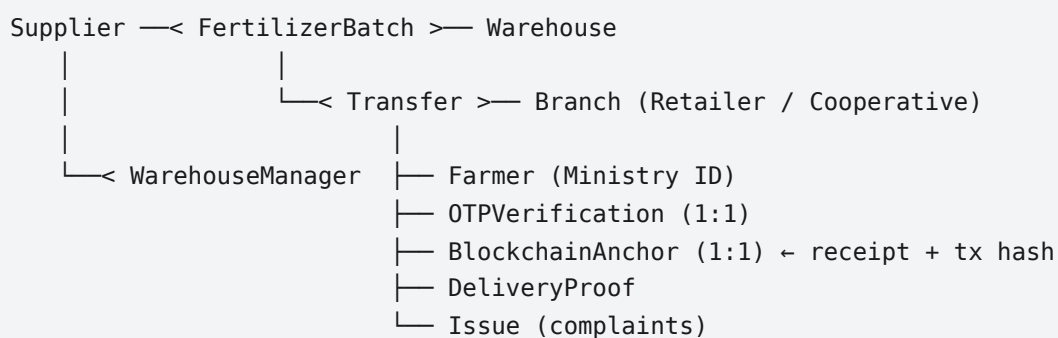
SEED_DEMO=true on deploy → seed_demo management command

Authorization Enforcement

- **Backend:** Permission classes on every viewset (`IsAdmin`, `SupplierOrAdmin`, `RegulatorOrAdmin`, etc.)
- **Frontend:** Role-based routing to dashboards; regulator gets `readOnly: true`
- **Data scoping:** Querysets filtered by supplier/branch; receipt access checked per transfer

5. Domain Model

Core entities and relationships:



Key Models (`backend/supply\_chain/models.py`)

Model	Description
Supplier	Fertilizer supplier organization, linked to User
Warehouse	Physical storage location with capacity
WarehouseManager	User who approves dispatches for a supplier
Branch	Retailer shop or AMCOS cooperative
Farmer	Registered farmer with unique `ministry_id`
FertilizerBatch	Batch of fertilizer (code, type, quantity, certification)
Transfer	Movement of bags — supplier → branch or branch → farmer
OTPVerification	Farmer OTP session for a transfer
BlockchainAnchor	On-chain hash + receipt metadata (JSON payload)
Notification	In-app alerts per user
PendingRegistration	Self-service signup awaiting approval
AuditLog	Record of significant API actions

Transfer Types & Statuses

Types:

- `SUPPLIER\_TO\_BRANCH` — supplier dispatches to retailer/AMCOS
- `BRANCH\_TO\_FARMER` — branch distributes to farmer

Status lifecycle:

```
PENDING → DISPATCHED → RECEIVED → VERIFIED
      ↓
      REJECTED (warehouse rejection)
```

6. End-to-End Supply Chain Workflow

Phase 1: Supplier Creates Inventory

1. ****Supplier**** creates a `FertilizerBatch` (batch code, fertilizer type, quantity, warehouse).
2. Batch is tied to supplier and optional warehouse location.

Phase 2: Dispatch to Branch

1. **Supplier** creates a `Transfer` (`SUPPLIER\_TO\_BRANCH`, status `PENDING`).
2. Destination: retailer or cooperative branch.
3. **Warehouse manager** receives notification.
4. Manager **approves** → status `DISPATCHED`, or **rejects** with reason.
5. Destination branch notified of incoming delivery.

Phase 3: Branch Receives Stock

1. **Retailer or AMCOS** confirms receipt → `POST /api/transfers/{id}/receive/`.
2. Status → `RECEIVED`; `confirmed\_at` timestamp recorded.
3. Supplier and branch receive in-app notifications.

Phase 4: Distribution to Farmer

1. **Branch** creates `Transfer` (`BRANCH\_TO\_FARMER`) linking farmer + batch + quantity.
2. System validates stock (branch must have received bags first).
3. **Cooperative:** farmer must belong to that AMCOS.
4. **Retailer:** Ministry-verified buyers may receive subsidy discount (configurable %).
5. OTP automatically sent to farmer's phone via Briq Karibu.

Phase 5: Farmer Confirms (OTP)

1. Farmer receives SMS with verification code.
2. Branch staff enters OTP in the app → `POST /api/transfers/{id}/verify\_otp/`.
3. Briq verifies code; transfer status → `VERIFIED`.

Phase 6: Trust Record Created

1. System builds verification receipt JSON.
2. Receipt uploaded to Storacha (or backend fallback).
3. Cryptographic hash computed and anchored on Polygon Amoy.
4. `BlockchainAnchor` record saved with tx hash, CID, and receipt copy in DB.

7. Farmer Verification (OTP)

Farmers are **not app users**. Verification is proof-of-receipt, not login.

Step	Action
1	Branch creates farmer distribution transfer
2	`issue_distribution_otp()` calls Briq `/v1/otp/request`
3	Farmer receives SMS (or call/WhatsApp on resend)
4	Branch enters code → Briq `/v1/otp/verify`
5	On success → verification proof recorded + blockchain anchor

Configuration: `BRIQ_API_KEY`, `BRIQ_APP_KEY`, `OTP_CODE_LENGTH` (default 6), `OTP_EXPIRY_MINUTES` (default 10).

Local dev fallback: `BRIQ_OTP_LOCAL_FALLBACK=true` accepts any code when Briq is unavailable.

8. Trust Layer: Receipts, Storage & Blockchain

Receipt Contents

Each verification receipt JSON includes:

- Transfer ID, batch code, quantity, fertilizer type
- Farmer details (Ministry ID, name, phone)
- Branch/cooperative details
- Verified timestamp and verifying user
- Delivery proof CID (if photo uploaded)
- Integrity block: `data\_hash`, `payload\_signature`, `content\_cid`, `tx\_hash`, explorer URL

Storage Strategy (Priority Order)

1. Storacha / IPFS ← primary (decentralized, permanent)
2. Backend disk ← fallback cache (media/receipts/)
3. PostgreSQL ← durable fallback (`BlockchainAnchor.payload.receipt`)
4. IPFS gateway fetch ← load-time recovery if only CID known

Why DB copy matters: Render's filesystem is ephemeral — disk files are lost on redeploy. The PostgreSQL copy ensures new receipts survive redeployments.

Hash & Blockchain Pipeline

```
payload_signature = "{batch_code}|{quantity}|{content_cid}|{transfer_id}|{verified_at}"
data_hash         = SHA-256(payload_signature)
on-chain tx       = CoffeeChainAudit.anchorTransaction(transfer_id, data_hash)
```

Smart Contract (`CoffeeChainAudit.sol`)

```
function anchorTransaction(string transactionId, string hashValue) external {
    emit TransactionAnchored(transactionId, hashValue, block.timestamp);
}
```

- Stores **only the hash**, not the full receipt (privacy + cost efficiency).
- Full receipt remains in Storacha/backend/DB.
- Verify on [Polygonscan Amoy](https://amoy.polygonscan.com) → Logs tab → `TransactionAnchored` event.

9. Data Integrity & Tamper Detection

Manual Integrity Checks

Admin/regulator uses the **Integrity panel** to:

- List all verified transfers
- Compare database fields vs stored receipt
- Compare DB hash vs receipt hash vs computed hash
- View source of tampering (app user vs direct database edit)

API: GET /api/integrity/ , POST /api/integrity/scan/

Automatic Tamper Detection

Django signal on `Transfer` `post_save` (`signals.py`):

- Monitors changes to anchored transfers (quantity, batch, farmer, status)
- Runs integrity check in background
- On mismatch → in-app notification + SMS to admins

Integrity Result Statuses

Status	Meaning
`ok`	Database matches receipt and hash
`mismatch`	Fields or hash differ — possible tampering
`missing_receipt`	Anchor exists but receipt not found
`missing_anchor`	Transfer not yet verified on chain
`unchecked`	Not yet compared

10. Notifications

In-App Notifications

Stored in `Notification` model; polled every 45 seconds by frontend.

Type	Trigger examples
`dispatch`	Supplier submitted dispatch
`receipt`	Branch confirmed receipt
`delivery`	Transfer dispatched
`otp`	OTP sent / verified
`registry`	Farmer registered
`system`	Integrity alert, registration request

SMS (Briq Karibu)

Use case	Recipients
Farmer OTP	Farmer phone on transfer
Registration approved/rejected	Applicant phone
Integrity tamper / restore	Admin phones (`UserProfile.contact_phone`)

11. Frontend Application

Routes

Path	Screen
`/`	Public landing page (bilingual EN/SW)
`/login`	Login form
`/signup`	Self-registration (supplier, retailer, cooperative)
`/app/{role}/{tab}`	Role dashboard

Dashboards by Role

Role	Component	Main tabs
Admin	`admin-dashboard.jsx`	Overview, Users, Registrations, Reports, Integrity, Profile
Regulator	`admin-dashboard.jsx` (read-only)	Same as admin, no edit actions
Supplier	`supplier-dashboard.jsx`	Overview, Dispatch, Dispatched, Warehouse, Issues, Analytics
Warehouse Manager	`warehouse-manager-dashboard.jsx`	Overview, Pending, Inventory, History, Analytics
Retailer	`retailer-dashboard.jsx`	Overview, Receive, Distribute, Customers, Verification, History
Cooperative	`cooperative-dashboard.jsx`	Overview, Farmers, Fertilizer In/Out, Verification, History

Key Frontend Modules

Path	Purpose
`frontend/src/app/api/client.js`	API client, JWT, token refresh
`frontend/src/app/components/integrity-panel.jsx`	Integrity monitoring UI
`frontend/src/app/components/verification-trust-seal.jsx`	Trust seal / receipt viewer
`frontend/src/app/utils/analytics-export.js`	CSV/PDF export helpers

12. Backend API Overview

Base URL: `{BACKEND_PUBLIC_URL}/api/`

Authentication

- `POST /login/`` — obtain JWT
- `POST /token/refresh/`` — refresh access token
- `GET/PATCH /me/`` — current user profile

Supply Chain

- `/suppliers/`, /branches/`, /warehouses/`, /batches/``
- `/transfers/`` — CRUD + workflow actions (approve, receive, verify_otp, receipt)
- `/farmers/`` — lookup, register (cooperative), resolve_buyer (retailer POS)

Governance

- `/auth/register/`` — public signup
- `/registrations/`` — pending signup review (admin)
- `/users/`` — user management (admin)

Trust & Audit

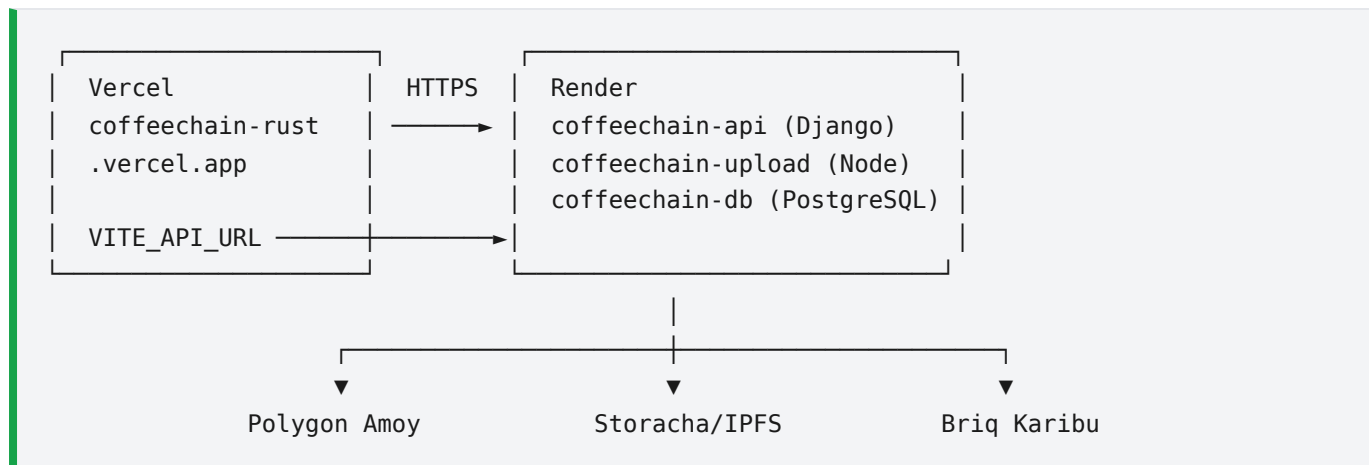
- `/integrity/`` — list, compare, batch scan
- `/anchors/`` — blockchain anchor records
- `/audit-logs/`` — action audit trail
- `/reports/`` — analytics and CSV export

Internal

- `/internal/receipt-callback/`` — upload-service receipt fallback
-

13. Deployment Architecture

Production Topology



Render Services (`render.yaml`)

Service	Name	Start command
Web (Python)	`coffeechain-api`	`bash start.sh` (migrate → backfill receipts → gunicorn)
Web (Node)	`coffeechain-upload`	`node index.js`
Database	`coffeechain-db`	PostgreSQL (free tier)

Deploy Checklist

Backend (Render):

- `DJANGO\_SECRET\_KEY`, `DJANGO\_ALLOWED\_HOSTS`, `BACKEND\_PUBLIC\_URL`
- `DATABASE\_URL` (auto from Render)
- `CORS\_ALLOWED\_ORIGINS=https://coffeechain-rust.vercel.app`
- `STORACHA\_UPLOAD\_ENABLED=true`, `STORACHA\_UPLOAD\_HOST` (auto-wired)
- `RECEIPT\_CALLBACK\_SECRET` (same on upload service)
- `POLYGON\_RPC\_URL`, `POLYGON\_CONTRACT\_ADDRESS`, `POLYGON\_PRIVATE\_KEY`
- `BRIQ\_API\_KEY`, `BRIQ\_APP\_KEY`

Upload service (Render):

- `STORACHA\_KEY`, `STORACHA\_PROOF`
- `BACKEND\_API\_HOST` (auto-wired), `RECEIPT\_CALLBACK\_SECRET`

Frontend (Vercel):

- `VITE\_API\_URL=https://coffeechain-api.onrender.com` (your API URL)

Smart contract (one-time):

```
cd smart-contracts && npm install && npm run compile && npm run deploy:amoy
# Set POLYGON_CONTRACT_ADDRESS in backend env
```

14. Environment Configuration

See `backend/.env.example`, `upload-service/.env.example`, and `render.yaml` for full lists.

Critical Variables

Variable	Service	Purpose
<code>`VITE_API_URL`</code>	Frontend	Backend API base URL
<code>`DATABASE_URL`</code>	Backend	PostgreSQL connection
<code>`STORACHA_UPLOAD_ENABLED`</code>	Backend	Enable Storacha uploads
<code>`STORACHA_KEY` / `STORACHA_PROOF`</code>	Upload	Storacha credentials
<code>`RECEIPT_CALLBACK_SECRET`</code>	Both	Secure receipt fallback
<code>`POLYGON_*`</code>	Backend	Blockchain anchoring
<code>`BRIQ_*`</code>	Backend	SMS OTP

15. Security Summary

Layer	Mechanism
User auth	JWT (access + refresh), bcrypt passwords
API access	Role-based permissions on every endpoint
Regulator	Read-only enforcement (backend + frontend)
Farmer proof	OTP via Briq — separate from user login
Receipt callback	Shared secret header, not public
CORS	Restricted to Vercel frontend origin in production
Integrity	Hash anchoring + continuous tamper monitoring
Audit	AuditLog for user actions; record modification tracking

16. Project Structure

```
CoffeeChain/
├── frontend/                # React SPA (Vercel)
│   ├── src/app/
│   │   ├── api/client.js    # API + JWT
│   │   ├── components/     # Dashboards, modals, UI
│   │   └── routes.jsx       # Router config
│   └── backend/            # Django API (Render)
│       ├── coffechain/     # Settings, root URLs
│       ├── supply_chain/   # Models, views, services
│       │   ├── models.py
│       │   ├── views.py
│       │   ├── permissions.py
│       │   └── services/    # ipfs, integrity, briq, polygon, notifications
│       └── start.sh         # Production startup
├── upload-service/         # Storacha upload (Render)
│   ├── index.js
│   ├── storacha-client.js
│   └── backend-callback.js
├── smart-contracts/        # Polygon Amoy contract
│   └── contracts/CoffeeChainAudit.sol
├── docs/
│   └── SYSTEM_ARCHITECTURE.md # This document
└── render.yaml             # Render blueprint
```

Appendix: Demo Workflow (Quick Test)

1. Login as ****supplier**** → create batch → dispatch to branch
 2. Login as ****warehouse manager**** → approve dispatch
 3. Login as ****retailer/cooperative**** → receive batch → distribute to farmer
 4. Enter farmer OTP → transfer becomes ****VERIFIED****
 5. Login as ****admin**** → Integrity tab → compare transfer → see hash match
 6. View receipt / trust seal → decentralized storage + Polygon link
-

Document generated for CoffeeChain team onboarding — Tanzania Coffee Board, 2026.

CoffeeChain — Tanzania Coffee Board · System Architecture · July 2026